

Full Stack Development

Paper Code: TMC 101 / TMD 111

Solved Question Paper with Detailed Answers

Course	MCA
Semester	1
Examination	Term Evaluation (Odd) Semester Examination, September 2025
Time / Max. Marks	1.5 hours / 50 marks
Pattern	5 questions x 10 marks. Each question has two options (a) or (b).

How to use this document

In the real exam you must attempt **any one** option (a or b) of each question. For revision, **both options of every question are solved here**, so you cover the whole paper.

Every answer is written to the marks allotted (10 marks each): definition, syntax, code, explanation, comparison tables, output and exam tips. In the exam you can trim the extra points, but the extra material is there to help you revise the full syllabus.

All code is written in HTML5 / CSS3 and can be typed into a file (for example `index.html`) and opened in any browser to see the result.

Contents

Q. No.	Topic (option a / option b)	CO
Q1	HTML and text formatting tags / div vs span	CO1
Q2	Types of CSS / HTML form with input controls	CO1, CO2
Q3	HTML table with colspan / Ordered, Unordered and Definition lists	CO1
Q4	CSS Box Model with labelled diagram / CSS selectors	CO2
Q5	Navigation menu with link states / Web page using ID, class, element, universal selectors	CO1, CO2
Appendix	Quick revision sheet: semantic tags, CSS properties, specificity, viva questions	-

- a. What is HTML? Explain any five Text Formatting Tags in HTML.
- b. Explain the differences between <div> and tags with suitable examples.

Answer (a): What is HTML? Five text formatting tags

1. What is HTML?

HTML (HyperText Markup Language) is the standard markup language used to create and structure web pages. The name explains itself:

- **HyperText** - text that contains links (hyperlinks) which take the reader to other pages or sections.
- **Markup** - special labels called tags that describe the structure and meaning of the content (heading, paragraph, list, image, table and so on).
- **Language** - a fixed set of rules and keywords that every browser understands.

HTML tells the browser *what* each piece of content is. The look is added by CSS and the behaviour by JavaScript. Together these three are the core of front-end web development.

Important points about HTML

- Created by **Tim Berners-Lee** at CERN in 1991. The current version is **HTML5**, which is maintained today as the HTML Living Standard.
- HTML is a **markup language, not a programming language** because it has no loops, conditions or variables. It only describes content.
- A page is a plain text file saved with the extension **.html** or **.htm** and opened in a browser (Chrome, Firefox, Edge, Safari).
- Tags are written inside angle brackets, for example **<p>**. Most tags come in pairs (**<p> ... </p>**). Tags without content are **empty (void) elements** such as **
, **<hr>, ****, **<input>**, **<meta>**.
- Tag names are **not case sensitive** (**<P>** and **<p>** work alike) but lower case is the recommended style.
- HTML5 added semantic tags (**<header>**, **<nav>**, **<section>**, **<article>**, **<footer>**), built-in audio and video (**<audio>**, **<video>**), **<canvas>**, and new form input types.
- It is platform independent, free, easy to learn, and supported by all browsers.

Basic structure of an HTML document

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is my first web page.</p>
</body>
</html>
```

Part	Purpose
<!DOCTYPE html>	Declares that the document is HTML5, so the browser uses standards mode.
<html>	Root element. It wraps the whole page. lang="en" sets the page language.

Part	Purpose
<head>	Holds information about the page (not shown in the page area): title, meta tags, CSS links, scripts.
<title>	Text shown on the browser tab and used by search engines.
<meta charset>	Sets the character encoding (UTF-8) so all symbols and languages display correctly.
<body>	Contains everything visible: text, images, links, tables, forms, etc.

Anatomy of an HTML element

<code><p class="intro">Welcome to HTML</p></code>	
	End (closing) tag
	Content
	Attribute value
	Attribute name
	Start (opening) tag
Start tag + content + end tag = one HTML ELEMENT	

2. Text formatting tags

Text formatting tags change how text looks or mark its meaning inside a page. They are of two kinds:

- **Physical tags** (`<b>`, `<i>`, `<u>`, `<sub>`, `<sup>`, `<small>`) only change appearance.
- **Logical / phrase (semantic) tags** (`<strong>`, `<em>`, `<mark>`, `<del>`, `<ins>`, `<code>`) also give meaning, which helps screen readers and search engines.

Five text formatting tags explained

Tag	Meaning and use	Syntax and example	Result
	Bold text. Draws attention without adding extra importance. (Its semantic partner <code></code> marks strong importance.)	<code>Note</code>	Note
<i>	Italic text. Used for technical terms, foreign words, thoughts. (Its partner <code></code> gives stress emphasis.)	<code><i>Python</i></code>	<i>Python</i>
<u>	Underlined text. Used for text that must look annotated, e.g. a misspelt word or a proper-name mark. Avoid using it for normal text as it looks like a link.	<code><u>Important</u></code>	<u>Important</u>
<sub>	Subscript - text set below the baseline. Used in chemical formulas and indices.	<code>H<sub>2</sub>O</code>	H ₂ O
<sup>	Superscript - text set above the baseline. Used in powers, footnotes, dates.	<code>x<sup>2</sup></code>	x ²

Note: the last column is drawn with the real formatting so you can see the effect.

Example program using all five tags

```
<!DOCTYPE html>
<html>
<head><title>Text Formatting</title></head>
<body>
  <h2>Text Formatting Tags</h2>
  <p>This text is <b>bold</b>.</p>
  <p>This text is <i>italic</i>.</p>
  <p>This text is <u>underlined</u>.</p>
  <p>Water is written as H<sub>2</sub>O.</p>
  <p>Pythagoras: a<sup>2</sup> + b<sup>2</sup> = c<sup>2</sup></p>
</body>
</html>
```

Output in the browser Text Formatting Tags

This text is **bold**.

This text is *italic*.

This text is underlined.

Water is written as H₂O.

Pythagoras: $a^2 + b^2 = c^2$

Other useful formatting tags (extra points for revision)

Tag	Purpose	Example
	Strong importance (usually bold)	Warning
	Emphasised text (usually italic)	really good
<mark>	Highlighted / marked text (yellow background)	<mark>exam</mark>
<small>	Smaller text (fine print, copyright)	T&C apply
	Deleted text (strike-through)	Rs 500
<ins>	Inserted text (underlined)	<ins>Rs 400</ins>
<code>	Computer code (monospace)	<code>x = 10;</code>
<pre>	Preformatted text - keeps spaces and line breaks	<pre>a b</pre>
<abbr>	Abbreviation with tooltip via title	<abbr title="HyperText Markup Language">HTML</abbr>
<blockquote> / <q>	Long quotation (block) / short quotation (inline)	<q>To be or not to be</q>

Exam tip

Write the definition of HTML, three or four key points, the basic structure, and then five tags each with syntax, purpose and an example. This covers the full 10 marks. Mention that **** and **<i>** are for looks while **** and **** carry meaning.

Answer (b): Differences between <div> and

1. Introduction

Both `<div>` and `<span>` are **generic, non-semantic container elements**. They have no meaning and no special look of their own. Their job is to **group content** so that it can be styled with CSS or controlled with JavaScript, usually through the `class` or `id` attribute.

- `<div>` (short for *division*) is a **block-level** container used to group large parts of a page, mainly for layout.
- `<span>` is an **inline** container used to mark a small piece of text inside a line, mainly for styling.

2. Differences between div and span

Basis	<div>	
Type of element	Block-level element	Inline element
Default display	<code>display: block;</code>	<code>display: inline;</code>
Line break	Always starts on a new line and the next element also goes to a new line.	Does not break the line. It stays in the same line as the surrounding text.
Width	Takes the full available width of the parent by default.	Takes only the width of its content .
Width, height, top/bottom margin	Can be set and are respected.	Ignored (until it is changed to <code>inline-block</code> or <code>block</code>).
Can contain	Block and inline elements: <code><p></code> , <code><h1></code> , <code></code> , <code><table></code> , other <code><div></code> s, <code></code> s, text.	Only text and other inline elements (<code><a></code> , <code></code> , <code></code> , other <code></code> s). It should not contain block elements.
Main purpose	Layout and grouping of sections: header, sidebar, card, wrapper, footer.	Styling a part of a line: colour a word, highlight text, add an icon.
Alignment	<code>text-align</code> and <code>margin: 0 auto</code> can align the box.	Follows the alignment of its parent line.
Typical usage	Container for many elements.	Small portion of text.
Semantics	None (use <code><section></code> , <code><article></code> when meaning is needed).	None.

3. Example of div (block behaviour)

```
<div style="background-color:lightblue; padding:10px;">
  <h2>Heading inside div</h2>
  <p>This paragraph is inside the first div.</p>
</div>
<div style="background-color:lightgreen; padding:10px;">
  Second div starts on a new line and takes full width.
</div>
```

Output

Two full-width coloured boxes, one below the other. The first box holds a heading and a paragraph; the second box holds one line of text.

4. Example of span (inline behaviour)

```
<p>My favourite colour is  
  <span style="color:red; font-weight:bold;">red</span>  
  and I love  
  <span style="background-color:yellow;">coding</span>.  
</p>
```

Output

My favourite colour is **red** and I love **coding**. (all in one line)

5. Using div and span together with CSS classes

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
  .card { width:300px; border:2px solid navy; padding:12px; margin:10px auto; }  
  .price { color:green; font-weight:bold; }  
  .old { color:red; text-decoration:line-through; }  
</style>  
</head>  
<body>  
  <div class="card">  
    <h3>Laptop</h3>  
    <p>Price: <span class="price">Rs 45,000</span>  
      <span class="old">Rs 50,000</span></p>  
  </div>  
</body>  
</html>
```

Here the **<div>** creates the card (a block box with border and width), while the two **** elements only colour parts of the sentence.

6. Changing the default behaviour with CSS

```
div { display: inline; } /* div now behaves like a span */  
span { display: block; } /* span now behaves like a div */  
span { display: inline-block; width: 100px; height: 40px; } /* inline + size */
```

7. Points to remember

- Use **<div>** when you need a **box** or a **section**; use **** when you need to style **words inside a line**.
- Placing a **<div>** inside a **** is invalid HTML; the reverse is fine.
- A **<div>** cannot be placed inside a **<p>** because a paragraph accepts only inline content.
- For meaningful structure prefer semantic tags (**<header>**, **<nav>**, **<main>**, **<section>**, **<footer>**) and keep **<div>** and **** for cases where no suitable semantic tag exists.

- a. What are the different types of CSS? Explain each with syntax and example.
- b. Write the HTML code to create a form with text box, radio button, checkbox, and submit button.

Answer (a): Types of CSS

1. What is CSS?

CSS (Cascading Style Sheets) is a style sheet language used to describe the presentation of an HTML document: colours, fonts, spacing, layout, borders, backgrounds and animations. It separates **content (HTML)** from **design (CSS)**, so one style sheet can control many pages. The word *cascading* means that when several rules apply to the same element, the browser decides the winner by a fixed order of priority.

CSS syntax

```
selector {  
  property: value;  
  property: value;  
}  
  
h1 {  
  color: blue;           /* h1      -> selector (what to style)      */  
  font-size: 30px;       /* color   -> property, blue -> value          */  
                          /* each property: value pair is a declaration */  
}                        /* everything between { } is the declaration block */
```

There are **three ways (types)** of adding CSS to an HTML page: **Inline**, **Internal (Embedded)** and **External** CSS.

Type 1: Inline CSS

CSS is written directly inside the HTML tag using the **style** attribute. It affects only that one element.

```
Syntax:  
<tagname style="property1: value1; property2: value2;">Content</tagname>  
  
Example:  
<!DOCTYPE html>  
<html>  
<body>  
  <h1 style="color:blue; text-align:center;">Inline CSS Heading</h1>  
  <p style="color:green; font-size:18px;">This paragraph is styled inline.</p>  
</body>  
</html>
```

- **Advantages:** quick for a single element or a quick test; highest priority; no separate file or selector needed.
- **Disadvantages:** cannot be reused, mixes content with design, hard to maintain, makes the HTML long and untidy.

Type 2: Internal (Embedded) CSS

CSS is written inside a `<style>` element placed in the `<head>` section of the same HTML file. It can style many elements of that one page using selectors.

```
Syntax:
<head>
  <style>
    selector { property: value; }
  </style>
</head>

Example:
<!DOCTYPE html>
<html>
<head>
  <style>
    body { background-color: #f0f8ff; }
    h1   { color: navy; text-align: center; }
    p    { color: darkgreen; font-size: 16px; }
  </style>
</head>
<body>
  <h1>Internal CSS Heading</h1>
  <p>All paragraphs of this page get the same style.</p>
  <p>This is another paragraph.</p>
</body>
</html>
```

- **Advantages:** one place for the whole page, can use class and id selectors, no extra file request.
- **Disadvantages:** applies to that page only, so it cannot be shared across a website; makes the file larger.

Type 3: External CSS

CSS is written in a separate file saved with the extension `.css`. The HTML page connects to it with a `<link>` tag placed in the `<head>`. This is the **best and most professional method** for real websites.

```
Syntax (in HTML):
<head>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>

File: style.css                                File: index.html
-----
body { background: #fafafa; }                  <!DOCTYPE html>
h1   { color: crimson; }                      <html>
p    { font-size: 17px; }                     <head>
                                              <link rel="stylesheet" href="style.css">
                                              </head>
                                              <body>
                                                <h1>External CSS</h1>
                                                <p>Styled from style.css</p>
                                              </body>
                                              </html>
```

An external file can also be attached from inside a `<style>` block using the `@import` rule: `@import url("style.css");`.

- **Advantages:** one file controls the look of the whole website; reusable; clean HTML; browser caches the file so pages load faster; easy maintenance.
- **Disadvantages:** an extra HTTP request; page may show unstyled for a moment if the file loads slowly or is missing.

Comparison of the three types

Basis	Inline CSS	Internal CSS	External CSS
Where written	<code>style</code> attribute inside the tag	<code><style></code> tag inside <code><head></code>	Separate <code>.css</code> file
Linked using	Not needed	Not needed	<code><link></code> tag or <code>@import</code>
Scope	One element	One web page	Many pages / whole website
Uses selectors?	No	Yes	Yes
Reusability	None	Within one page	Highest
Maintenance	Very hard	Moderate	Easy
Priority	Highest	Middle (depends on order with external)	Lowest of the three (depends on order)
Best used for	Quick single change	Single page with unique style	Real, multi-page websites

Cascading order (which style wins?)

1. Inline style (highest, unless `!important` is used elsewhere).
2. Internal and external styles have equal weight, so the **one written later** in the document wins.
3. Browser default style (lowest).

If two rules of the same type conflict, the one with the **higher specificity** wins (id > class > element). A declaration marked `!important` overrides normal rules.

Exam tip

Always draw or write the three headings clearly and give one small code example for each. Adding the comparison table gives a full-marks answer.

OR (Option b)

Answer (b): HTML form with text box, radio button, checkbox and submit button

1. HTML form basics

An HTML **form** collects input from the user and sends it to a server. It is created with the `<form>` tag. Input controls are placed inside it, mostly with the `<input>` tag whose `type` attribute decides the control.

2. Program

```
<!DOCTYPE html>
<html>
<head>
  <title>Registration Form</title>
</head>
<body>
  <h2>Student Registration Form</h2>

  <form action="submit.php" method="post">

    <!-- Text box -->
    <label for="uname">Name:</label>
```

```

<input type="text" id="uname" name="username"
      placeholder="Enter your name" required>
<br><br>

<!-- Radio buttons (same name = only one can be chosen) -->
<p>Gender:</p>
<input type="radio" id="male" name="gender" value="male" checked>
<label for="male">Male</label>
<input type="radio" id="female" name="gender" value="female">
<label for="female">Female</label>
<br><br>

<!-- Check boxes (many can be chosen) -->
<p>Hobbies:</p>
<input type="checkbox" id="h1" name="hobby" value="reading">
<label for="h1">Reading</label>
<input type="checkbox" id="h2" name="hobby" value="music">
<label for="h2">Music</label>
<input type="checkbox" id="h3" name="hobby" value="sports">
<label for="h3">Sports</label>
<br><br>

<!-- Submit button -->
<input type="submit" value="Submit">
</form>
</body>
</html>

```

3. Output (how the form looks in the browser)

Student Registration

Name:

Gender: ☒ Male ☐ Female

Hobbies: ☒ Reading ☐ Music

4. Explanation of the code

Element / attribute	Explanation
<code><form></code>	Container of all controls. action gives the page/script that receives the data; method gives how it is sent (get or post).
<code><label for="id"></code>	Text caption of a control. Clicking the label selects the control. The for value must equal the control's id .
<code>type="text"</code>	Single-line text box . placeholder shows a hint, required makes the field compulsory, maxlength limits characters.
<code>type="radio"</code>	Radio button to choose exactly one option. All buttons of one group must have the same name ; value is what gets sent; checked selects by default.
<code>type="checkbox"</code>	Check box to choose zero, one or many options. Each can be ticked independently.
<code>type="submit"</code>	Submit button . Clicking it sends the form data to the address in action . value is the text shown on the button.

Element / attribute	Explanation
<code>name</code>	Key under which the value is sent to the server. Without <code>name</code> the data is not submitted.
<code>id</code>	Unique identifier used by <code><label></code> , CSS and JavaScript.

5. GET vs POST

Basis	GET	POST
Data location	Appended to the URL (visible)	Sent in the request body (not visible)
Security	Less secure	More secure
Data size	Limited by URL length	Practically unlimited
Use for	Searching, bookmarking	Login, registration, uploads

6. Other input types and controls (for revision)

Control	Code	Purpose
Password	<code><input type="password"></code>	Hides typed characters
Email	<code><input type="email"></code>	Validates e-mail format
Number	<code><input type="number" min="1" max="10"></code>	Numeric input with limits
Date	<code><input type="date"></code>	Date picker
File	<code><input type="file"></code>	Upload a file
Reset	<code><input type="reset"></code>	Clears the form
Button	<code><button type="submit">Send</button></code>	Flexible button element
Drop-down list	<code><select><option>..</option></select></code>	Choose from a list
Text area	<code><textarea rows="4" cols="30"></textarea></code>	Multi-line text
Fieldset	<code><fieldset><legend>..</legend></fieldset></code>	Groups related controls

Exam tip

Remember: radio buttons of one group share the same `name`; check boxes may share a name too but allow many selections; the submit button needs a form with an `action` to send data.

- a. Write HTML code to create a table with 3 rows and 4 columns, where the second cell of the first row spans 2 columns.
- b. Differentiate between Ordered List, Unordered List, and Definition List in HTML. Provide examples.

Answer (a): Table with 3 rows and 4 columns (colspan)

1. Concept

An HTML table arranges data in rows and columns. It is built with `<table>`; each row is `<tr>` (table row); each normal cell is `<td>` (table data) and each heading cell is `<th>` (table header).

The attribute `colspan` makes one cell stretch across several **columns** (horizontal merge). `colspan="2"` means the cell occupies the space of two columns. Because that cell uses two column slots, the **first row needs only 3 `<td>` tags** to fill the 4 columns:

Row 1 :	1 (Cell 1) + 2 (Cell 2, colspan="2") + 1 (Cell 3)	= 4 columns -> 3 <code><td></code> tags
Row 2 :	1 + 1 + 1 + 1	= 4 columns -> 4 <code><td></code> tags
Row 3 :	1 + 1 + 1 + 1	= 4 columns -> 4 <code><td></code> tags

2. Program

```
<!DOCTYPE html>
<html>
<head>
  <title>Table with Colspan</title>
  <style>
    table { border-collapse: collapse; width: 80%; }
    td    { border: 1px solid black; padding: 10px; text-align: center; }
  </style>
</head>
<body>
  <h2>Table with 3 Rows and 4 Columns</h2>
  <table>
    <!-- Row 1 : second cell spans 2 columns -->
    <tr>
      <td>Cell 1</td>
      <td colspan="2">Cell 2 (colspan = 2)</td>
      <td>Cell 3</td>
    </tr>
    <!-- Row 2 -->
    <tr>
      <td>Cell 4</td>
      <td>Cell 5</td>
      <td>Cell 6</td>
      <td>Cell 7</td>
    </tr>
    <!-- Row 3 -->
    <tr>
      <td>Cell 8</td>
      <td>Cell 9</td>
      <td>Cell 10</td>
      <td>Cell 11</td>
    </tr>
  </table>
</body>
</html>
```

Without CSS you can also write the simple form `<table border="1">` to get visible borders.

3. Output

Cell 1	Cell 2 (colspan = 2)		Cell 3
Cell 4	Cell 5	Cell 6	Cell 7
Cell 8	Cell 9	Cell 10	Cell 11

The yellow cell is "Cell 2" stretched across the 2nd and 3rd columns of the first row.

4. Table tags and attributes to remember

Tag / attribute	Meaning
<code><table></code>	Creates the table.
<code><tr></code>	Table row.
<code><td></code>	Table data cell (normal cell).
<code><th></code>	Table header cell (bold and centred by default).
<code><caption></code>	Title of the table.
<code><thead></code> , <code><tbody></code> , <code><tfoot></code>	Group header rows, body rows and footer rows.
<code>colspan="n"</code>	Cell spans n columns (horizontal merge).
<code>rowspan="n"</code>	Cell spans n rows (vertical merge).
<code>border</code> , <code>cellpadding</code> , <code>cellspacing</code>	Old attributes for border, inner space and gap. Now done with CSS (<code>border</code> , <code>padding</code> , <code>border-collapse</code>).

5. Extra: rowspan example

```
<table border="1">
  <tr>
    <td rowspan="2">Spans 2 rows</td>
    <td>Row 1, Col 2</td>
  </tr>
  <tr>
    <td>Row 2, Col 2</td>
  </tr>
</table>
```

Exam tip

The main mark-earning point is `colspan="2"` on the second `<td>` of the first row and writing only three cells in that row. Show the complete program and draw the output grid.

Answer (b): Ordered, Unordered and Definition lists

1. Introduction

HTML **lists** group related items. HTML provides three kinds of lists: **Ordered list** (`<ol>`), **Unordered list** (`<ul>`) and **Definition (description) list** (`<dl>`).

2. Ordered List (`<ol>`)

Items appear in a **sequence** and are marked with **numbers or letters**. Each item is written in `<li>` (list item). Use it when order matters, e.g. steps of a recipe or ranking.

```
<ol>
  <li>Open the editor</li>
  <li>Write the code</li>
  <li>Save the file</li>
</ol>

<!-- Attributes -->
<ol type="A" start="3">    <!-- letters A, B, C... starting from C -->
<ol type="i">              <!-- small roman numerals -->
<ol reversed>             <!-- counts down: 3, 2, 1 -->
```

type values: **1** (numbers, default), **A** (capital letters), **a** (small letters), **I** (capital roman), **i** (small roman).
start gives the first number.

3. Unordered List (`<ul>`)

Items are **not in any order** and are marked with **bullets**. Each item uses `<li>`. Use it for a collection such as a shopping list or a menu.

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

/* Bullet style is changed with CSS */
ul { list-style-type: square; }    /* disc | circle | square | none */
```

4. Definition List (`<dl>`)

A list of **terms and their descriptions**. It uses three tags: `<dl>` (list), `<dt>` (definition term) and `<dd>` (definition description, shown indented). Use it for glossaries and FAQs.

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language - structures a web page.</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets - styles a web page.</dd>
</dl>
```

5. Differences

Basis	Ordered List	Unordered List	Definition List
Main tag	<code></code>	<code></code>	<code><dl></code>
Item tag	<code></code>	<code></code>	<code><dt></code> (term) and <code><dd></code> (description)
Marker	Numbers, letters, roman numerals	Bullets: disc, circle, square	No marker; description is indented
Order of items	Order is important	Order is not important	Term followed by its description
Attributes	<code>type</code> , <code>start</code> , <code>reversed</code>	Style via CSS <code>list-style-type</code>	None special (style with CSS)
Structure	One item per <code></code>	One item per <code></code>	Pairs of <code><dt></code> + <code><dd></code>
Used for	Steps, rankings, procedures	Menus, features, shopping lists	Glossary, FAQ, metadata
Default look	1. 2. 3.	Round bullets	Term, then indented meaning

6. Combined example and output

```
<h3>Ordered List</h3>
<ol>
  <li>Wake up</li>
  <li>Study</li>
  <li>Sleep</li>
</ol>
<h3>Unordered List</h3>
<ul>
  <li>Apple</li>
  <li>Banana</li>
</ul>
<h3>Definition List</h3>
<dl>
  <dt>Browser</dt>
  <dd>Software used to view web pages.</dd>
</dl>
```

Output

Ordered List

1. Wake up
2. Study
3. Sleep

Unordered List

- Apple
- Banana

Definition List

Browser

Software used to view web pages.

7. Nested lists

A list can be placed inside an `<li>` of another list to make sub-levels.

```
<ul>
  <li>Web Technologies
    <ul>
      <li>HTML</li>
      <li>CSS</li>
    </ul>
  </li>
  <li>Databases</li>
</ul>
```

Question 4

10 Marks | CO2

- a. Describe the structure and components of the CSS Box Model with a labeled diagram.
- b. What is CSS selectors? Explain Element Selector, Id Selector and Class Selector with example.

Answer (a): CSS Box Model

1. Introduction

In CSS, **every HTML element is treated as a rectangular box**. The **CSS Box Model** is the set of rules that describes how the size and space of this box are calculated. It consists of four layers, from the inside to the outside: **Content**, **Padding**, **Border** and **Margin**. Understanding it is essential for layout, spacing and alignment.

2. Labelled diagram

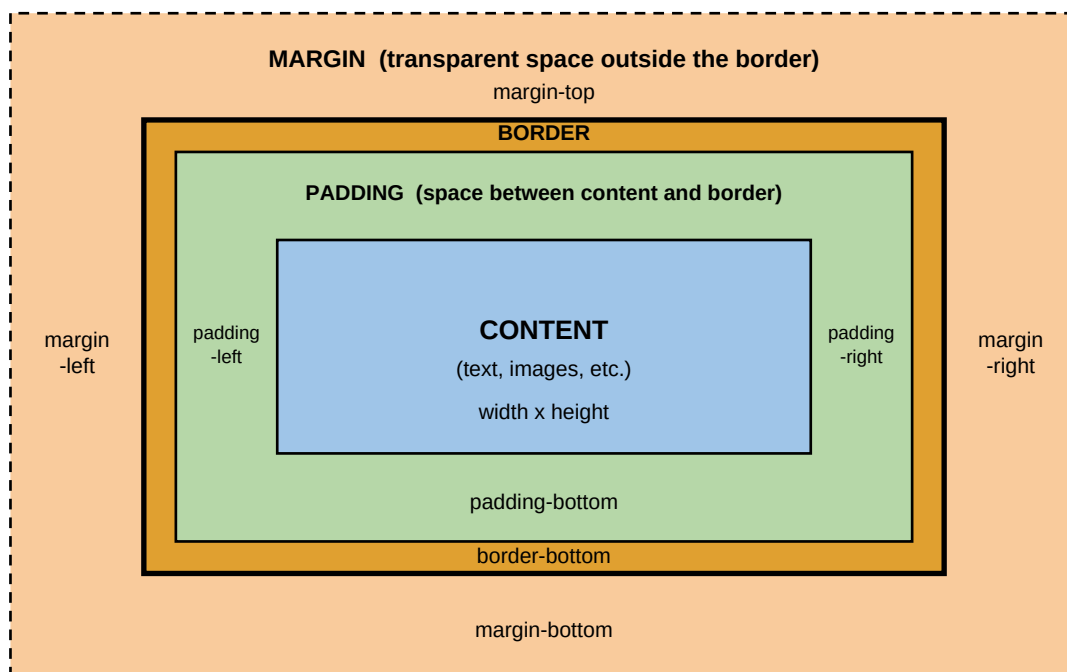


Fig: CSS Box Model (Content, Padding, Border, Margin from inside to outside)

3. Components of the box model

Component	Description	CSS properties	Notes
Content	The innermost area where the actual text, images or child elements are shown.	<code>width</code> , <code>height</code> , <code>min-width</code> , <code>max-width</code>	Element background shows here.
Padding	Transparent space between the content and the border (inside the box).	<code>padding</code> , <code>padding-top</code> , <code>padding-right</code> , <code>padding-bottom</code> , <code>padding-left</code>	Takes the element background colour. Cannot be negative.
Border	The line that surrounds the padding and content.	<code>border</code> , <code>border-width</code> , <code>border-style</code> , <code>border-color</code> , <code>border-radius</code>	Shorthand: <code>border: 2px solid black;</code>
Margin	Transparent space outside the border , separating the element from its neighbours.	<code>margin</code> , <code>margin-top</code> , <code>margin-right</code> , <code>margin-bottom</code> , <code>margin-left</code>	Always transparent. Can be negative. <code>margin: 0 auto;</code> centres a block.

4. Shorthand values (clockwise from top)

```
margin: 10px;           /* all four sides = 10px           */
margin: 10px 20px;      /* top+bottom = 10px, left+right = 20px          */
margin: 10px 20px 30px; /* top = 10, left+right = 20, bottom = 30        */
margin: 10px 20px 30px 40px; /* top, right, bottom, left (clockwise)      */
/* padding and border-width use exactly the same pattern */
```

5. Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .box {
      width: 200px;
      height: 100px;
      padding: 20px;
      border: 5px solid navy;
      margin: 10px;
      background-color: lightblue;
    }
  </style>
</head>
<body>
  <div class="box">Box Model Demo</div>
</body>
</html>
```

6. Calculating the total size

```
Total width  = width + padding-left + padding-right + border-left + border-right
               + margin-left + margin-right
Total height = height + padding-top + padding-bottom + border-top + border-bottom
               + margin-top + margin-bottom
```

For the example above	Calculation	Result
Box width (without margin)	$200 + (20 + 20) + (5 + 5)$	250 px
Total width space (with margin)	$250 + (10 + 10)$	270 px
Box height (without margin)	$100 + (20 + 20) + (5 + 5)$	150 px
Total height space (with margin)	$150 + (10 + 10)$	170 px

7. box-sizing property

Value	Meaning	Example (width: 200px; padding: 20px; border: 5px)
content-box (default)	width and height apply to the content only . Padding and border are added outside it.	Content = 200 px, visible box = 250 px wide
border-box	width and height include content + padding + border. Easier to plan layouts.	Visible box = 200 px, content shrinks to 150 px

```
* { box-sizing: border-box; } /* commonly used at the top of a style sheet */
```

8. Margin vs Padding

Basis	Padding	Margin
Position	Inside the border, around the content	Outside the border, around the element
Background	Shows the element background colour	Always transparent
Purpose	Space between content and border	Space between one element and another
Negative value	Not allowed	Allowed
Auto value	Not allowed	margin: auto centres block elements

9. Margin collapsing

When the **vertical margins of two adjacent block elements meet**, they **collapse** into one: the gap equals the larger margin, not the sum. Example: bottom margin 30px of one box and top margin 20px of the next give a gap of **30px** (not 50px). Horizontal margins never collapse.

Exam tip

Draw the diagram with four nested rectangles and label Content, Padding, Border and Margin. Write the total width formula and one numerical example. Mention **box-sizing**.

OR (Option b)

Answer (b): CSS Selectors

1. What is a CSS selector?

A **CSS selector** is the part of a CSS rule that **selects the HTML element(s) to be styled**. It appears before the curly braces. The declarations inside the braces are applied to every element that matches the selector.

```
selector {  
    property: value;  
}  
/* Example */  
p { color: red; } /* selector = p -> every <p> becomes red */
```

Main types of selectors (overview)

Selector	Syntax	Selects
Universal	<code>* { }</code>	Every element
Element (type)	<code>p { }</code>	All elements of that tag name
Id	<code>#main { }</code>	The one element with <code>id="main"</code>
Class	<code>.note { }</code>	All elements with <code>class="note"</code>
Group	<code>h1, h2, p { }</code>	Several selectors sharing one rule
Descendant	<code>div p { }</code>	All <code><p></code> inside a <code><div></code>
Child	<code>div > p { }</code>	<code><p></code> that are direct children of <code><div></code>
Attribute	<code>input[type="text"] { }</code>	Elements having that attribute value
Pseudo-class	<code>a:hover { }</code>	Elements in a special state
Pseudo-element	<code>p::first-line { }</code>	A part of an element

2. Element selector (type selector)

Selects **all elements with the given tag name**. No special symbol is used; just write the tag name.

```
Syntax:  elementname { property: value; }

p { color: blue; font-size: 16px; }    /* all paragraphs           */
h1 { text-align: center; }             /* all first-level headings  */
li { list-style-type: square; }        /* all list items            */
```

3. Id selector

Selects **one unique element** having the matching `id` attribute. It is written with a **hash (#)** followed by the id name. An id must be **unique in a page** and should be used for only one element.

```
Syntax:  #idname { property: value; }

CSS :    #mainHead { color: darkred; text-align: center; }
HTML :   <h1 id="mainHead">Welcome</h1>
```

4. Class selector

Selects **all elements having the given class attribute**. It is written with a **dot (.)** followed by the class name. A class can be **reused on many elements** and one element can have **several classes** separated by spaces.

```
Syntax:  .classname { property: value; }

CSS :    .note { background-color: yellow; padding: 5px; }
        p.note { font-style: italic; }                    /* only <p> having class note */
HTML :   <p class="note">First note</p>
        <div class="note">Second note</div>
        <p class="note big">Two classes: note and big</p>
```

5. Complete example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    p      { color: navy; }           /* element selector */
    #title { color: crimson; text-align: center; } /* id selector   */
    .highlight { background-color: yellow; } /* class selector  */
  </style>
</head>
<body>
  <h1 id="title">CSS Selectors</h1>
  <p>Normal paragraph (element selector).</p>
  <p class="highlight">Highlighted paragraph (class selector).</p>
  <p class="highlight">Another highlighted paragraph.</p>
</body>
</html>
```

6. Comparison

Basis	Element selector	Id selector	Class selector
Symbol	None (tag name)	# (hash)	. (dot)
HTML attribute needed	No	id="..."	class="..."
Selects	All elements of that tag	Only one unique element	Any number of elements
Reuse on many elements	Automatic for that tag	No (must be unique)	Yes
Specificity	Lowest (0,0,1)	Highest (1,0,0)	Middle (0,1,0)
Example	p { }	#mainHead { }	.para { }

7. Specificity (which rule wins?)

If several rules target the same element, the one with the **higher specificity** wins: **inline style > id > class (and attribute, pseudo-class) > element**. Equal specificity: the rule written last wins.

```
p      { color: black; } /* weight 1   */
.para   { color: green; } /* weight 10  */
#intro  { color: red; }  /* weight 100 */
<p id="intro" class="para">Text</p>    -> the text is RED (id is strongest)
```

- a. Create a navigation menu with 3 links (Home, About, Contact). Normal state: text should be black. On hover: text should turn red and underline. On visited: text should turn green.
- b. Design a web page with: a heading styled using an ID selector (e.g., #mainHead); a paragraph styled using a class selector (e.g., .para); a list styled using an element selector; and a universal selector to set margin and padding for all elements.

Answer (a): Navigation menu with link states

1. Concept: link pseudo-classes

A hyperlink is created with the anchor tag `<a href="...">`. A link can be in different **states**, and CSS styles each state with a **pseudo-class**:

State	Pseudo-class	When it applies	Required style
Normal (unvisited)	<code>a:link</code>	Link that has not been visited yet	Text black
Visited	<code>a:visited</code>	Link the user has already opened	Text green
Hover	<code>a:hover</code>	Mouse pointer is over the link	Text red and underline
Active	<code>a:active</code>	Link is being clicked	(optional) orange

Very important rule - LVHA order

The rules must be written in this order: **a:link**, **a:visited**, **a:hover**, **a:active** ("LoVe HAtE"). Reason: all four have the same specificity, so the later rule wins. If **a:hover** is written before **a:visited**, a visited link will stay green and never turn red on hover.

2. Program

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Navigation Menu</title>
  <style>
    nav ul {
      list-style-type: none;          /* remove bullets          */
      margin: 0;
      padding: 0;
      display: flex;                 /* place items in one horizontal row */
      background-color: #f1f1f1;
    }
    nav a {
      display: block;
      padding: 14px 24px;
      text-decoration: none;          /* no underline in normal state    */
    }
    a:link { color: black; }          /* normal */
    a:visited { color: green; }       /* visited */
    a:hover { color: red; text-decoration: underline; } /* hover */
    a:active { color: orange; }       /* clicked */
  </style>
</head>
```

```

<body>
  <nav>
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="about.html">About</a></li>
      <li><a href="contact.html">Contact</a></li>
    </ul>
  </nav>
</body>
</html>

```

3. Output

Home	About (visited link)	Contact (mouse over it)
------	----------------------	-------------------------

A grey horizontal bar with the three links. Unvisited links are black; a link already opened is green; the link under the mouse turns red with an underline.

4. Explanation

- `<nav>` is the HTML5 semantic tag for a block of navigation links. The links are kept in an unordered list.
- `list-style-type: none`, `margin: 0` and `padding: 0` remove the default bullets and space of the list. `display: flex` puts the items side by side.
- `text-decoration: none` removes the default underline so the underline appears **only on hover**.
- `a:link` gives the normal black colour, `a:visited` gives green, `a:hover` gives red with underline.
- To see the green colour, the pages must exist and be opened once; the browser remembers visited links.
- For privacy, browsers allow only colour-related properties inside `a:visited`.

Exam tip

Write the four pseudo-classes in LVHA order and state that order in one line. This is the point examiners look for.

OR (Option b)

Answer (b): Web page using ID, class, element and universal selectors

1. Plan

Requirement	Selector used	Code
Heading styled by ID	ID selector (#)	<code>#mainHead { ... }</code> with <code><h1 id="mainHead"></code>
Paragraph styled by class	Class selector (.)	<code>.para { ... }</code> with <code><p class="para"></code>
List styled by element	Element selector	<code>ul { ... }</code> and <code>li { ... }</code>
Margin and padding for all	Universal selector (*)	<code>* { margin: 0; padding: 0; }</code>

2. Program

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>CSS Selectors Demo</title>
  <style>
    /* 1. Universal selector: applies to every element */
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }
  </style>

```

```

body {
  font-family: Arial, sans-serif;
  background-color: #f4f8fb;
  padding: 20px;
}

/* 2. ID selector: only the element with id="mainHead" */
#mainHead {
  color: darkblue;
  text-align: center;
  padding: 10px;
  margin-bottom: 15px;
  border-bottom: 3px solid darkblue;
}

/* 3. Class selector: every element with class="para" */
.para {
  color: #333333;
  font-size: 17px;
  line-height: 1.6;
  text-align: justify;
  margin-bottom: 15px;
}
/* 4. Element selectors for the list */
ul {
  margin-left: 30px;
}
li {
  color: darkgreen;
  font-size: 16px;
  list-style-type: square;
  margin: 6px 0;
}
</style>
</head>
<body>
  <h1 id="mainHead">Full Stack Development</h1>

  <p class="para">Full stack development covers the front end, the back end and the
    database of a web application.</p>
  <p class="para">The front end is built with HTML, CSS and JavaScript.</p>

  <ul>
    <li>HTML - structure</li>
    <li>CSS - presentation</li>
    <li>JavaScript - behaviour</li>
  </ul>
</body>
</html>

```

3. Output

Web page as displayed

Full Stack Development

Full stack development covers the front end, the back end and the database of a web application.

The front end is built with HTML, CSS and JavaScript.

- HTML - structure
- CSS - presentation
- JavaScript - behaviour

4. Explanation

- **Universal selector** `*` matches every element on the page. Here it removes the default margin and padding of all browsers and sets `box-sizing: border-box` for predictable sizes.
- **ID selector** `#mainHead` styles only the heading that has `id="mainHead"`. An id can be used once per page.
- **Class selector** `.para` styles every element carrying `class="para"`, so both paragraphs share the same look.
- **Element selectors** `ul` and `li` style every list and list item without needing an id or class.
- Because the universal rule sets `padding: 0`, the list loses its default left indent, so `margin-left: 30px` is given to `ul` to bring the bullets back into view.
- The `body` rule overrides the universal padding for the body because an element selector is **more specific** than the universal selector.

5. Specificity of the selectors used

Selector	Specificity	Strength
<code>*</code> (universal)	(0,0,0)	Lowest - easily overridden
<code>ul, li, body</code> (element)	(0,0,1)	Low
<code>.para</code> (class)	(0,1,0)	Medium
<code>#mainHead</code> (id)	(1,0,0)	High
Inline <code>style=""</code>	(1,0,0,0)	Highest of these

Exam tip

Label each selector in a comment (`/* ID selector */` etc.). The examiner can then tick all four required selectors at a glance.

Appendix: Quick Revision Sheet

A. Block-level and inline elements

Type	Behaviour	Common elements
Block-level	Starts on a new line, takes full width	<code><div></code> , <code><p></code> , <code><h1></code> - <code><h6></code> , <code></code> , <code></code> , <code></code> , <code><table></code> , <code><form></code> , <code><header></code> , <code><footer></code> , <code><section></code> , <code><nav></code>
Inline	Stays in the line, takes only needed width	<code></code> , <code><a></code> , <code></code> , <code><i></code> , <code><u></code> , <code></code> , <code></code> , <code></code> , <code><input></code> , <code><label></code> , <code><sub></code> , <code><sup></code>

B. HTML5 semantic elements

Tag	Purpose
<code><header></code>	Introductory content or top area of a page/section
<code><nav></code>	Block of navigation links
<code><main></code>	Main unique content of the page
<code><section></code>	Thematic group of content
<code><article></code>	Independent, self-contained content (blog post, news)
<code><aside></code>	Side content (sidebar, ads)
<code><footer></code>	Bottom area: copyright, links

C. Links, images and media

Element	Syntax	Notes
Hyperlink	<code>Text</code>	<code>target="_blank"</code> opens a new tab
Image	<code></code>	Empty element; <code>alt</code> is shown if the image fails and is read by screen readers
Line break / rule	<code>
</code> and <code><hr></code>	Both are empty elements
Audio / video	<code><audio controls src="a.mp3"></audio></code>	HTML5 media, no plug-in needed
Comment	<code><!-- comment --></code>	Not displayed

D. Frequently used CSS properties

Group	Properties
Text	<code>color</code> , <code>font-family</code> , <code>font-size</code> , <code>font-weight</code> , <code>font-style</code> , <code>text-align</code> , <code>text-decoration</code> , <code>line-height</code> , <code>letter-spacing</code> , <code>text-transform</code>
Background	<code>background-color</code> , <code>background-image</code> , <code>background-repeat</code> , <code>background-size</code> , <code>background-position</code>
Box model	<code>width</code> , <code>height</code> , <code>margin</code> , <code>padding</code> , <code>border</code> , <code>border-radius</code> , <code>box-sizing</code>
Display / layout	<code>display</code> , <code>position</code> , <code>float</code> , <code>flex</code> , <code>grid</code> , <code>overflow</code> , <code>z-index</code>
Lists / tables	<code>list-style-type</code> , <code>border-collapse</code> , <code>border-spacing</code>

E. CSS units and colour formats

Item	Explanation
px	Absolute pixel unit
%	Relative to the parent element
em	Relative to the font size of the parent element
rem	Relative to the font size of the root (<html>) element
vw, vh	1% of viewport width / height
Colours	Name (red), hex (#ff0000), RGB (rgb(255, 0, 0)), HSL (hsl(0, 100%, 50%))

F. Important short questions and answers

Question	Answer
What is the difference between HTML and HTML5?	HTML5 is the latest version. It adds semantic tags, audio/video, canvas, new form input types and better APIs without needing plug-ins such as Flash.
What is an empty (void) element?	A tag with no content and no closing tag, e.g. , <hr>, , <input>, <meta>, <link>.
Difference between id and class?	An id is unique (#id) and used for one element; a class can be used by many elements (.class).
What is an attribute?	Extra information given inside the start tag as name="value", e.g. href, src, class, id, style.
What is the use of the alt attribute?	It gives alternative text for an image, used when the image does not load, by screen readers and by search engines.
Absolute vs relative path?	Absolute path gives the full URL (https://site.com/a.jpg); relative path is relative to the current file (images/a.jpg).
What are pseudo-classes?	Keywords added to a selector to style a special state, e.g. :hover, :visited, :focus, :first-child.
What does the viewport meta tag do?	It makes the page scale to the device width, which is the base of responsive design.
Inline vs block vs inline-block?	Inline: no line break and no width/height. Block: full width, line break. Inline-block: sits in the line but accepts width and height.
What is full stack development?	Building both the front end (HTML, CSS, JavaScript, React), the back end (Node.js, PHP, Java, Python) and the database (MySQL, MongoDB) of a web application.
What is the role of HTML, CSS and JavaScript?	HTML = structure, CSS = presentation, JavaScript = behaviour and interactivity.
Which CSS type has the highest priority?	Inline CSS (unless !important is used).

G. Common mistakes to avoid in the exam

- Forgetting `<!DOCTYPE html>` and the `<html>`, `<head>`, `<body>` structure.
- Writing four `<td>` in the colspan row of Q3 (it must be three).
- Giving different `name` values to radio buttons of one group.
- Using `#` for a class or `.` for an id.
- Writing `a:hover` before `a:visited` (wrong LVHA order).
- Not closing tags such as `</p>`, `</li>`, `</td>`, `</tr>`.
- Leaving out the diagram in the box model answer, or the comparison table in "differences" questions.

Best of luck!

Practise typing each program once in a text editor and open it in the browser. Seeing the output yourself is the fastest way to remember the syntax.